

RAG, CAG, Fine-Tuning, LangExtract și Ontologii

O perspectivă academică asupra integrării în procesarea limbajului
natural

conf.dr. Cristian KEVORCHIAN
Universitatea din Bucuresti

"Meaning is use"

Ludwig Wittgenstein - Philosophical Investigations(1953)

În **Philosophical Investigations**, atacă presupunerile viziunea Tractatus(1921) **Prefața din PI:**

"Am fost forțat să recunosc greșeli grave în ceea ce am scris în acea primă carte."

Tractatus = frumos dar greșit vs PI = pragmatic și adevărat.

Tractatus	Investigations
O singură esență a limbajului	Multiple "jocuri de limbaj"
Sensul = pictura realității	Sens = utilizare în practică
Limbaj ideal, logic	Limbaj natural, divers
Obiecte simple ca fundament	"Family resemblance"- conceptele nu au o esență comună care să unească toate instanțele, ci doar asemănări parțiale suprapuse, ca între membrii unei familii
Atomic facts independente	Sens contextual, holistic

	Wittgenstein (1953)	Solomon Marcus	LLM (Large Language Models)
Sensul cuvintelor	„Sensul este utilizare” în jocuri de limbaj	Sensul este relațional și dependent de context	Sensul este învățat din distribuția contextuală a cuvintelor în corpuri mari de text
Contextul	Contextul determină funcția și interpretarea	Contextul creează proximitate semantică	Contextul local (frază, paragraf) influențează predicția și generarea de text
Metafora	Nu o discută extensiv, dar acceptă variația sensului	Metafora ca mecanism de variație semantică	Metaforele sunt recunoscute și reproduse prin pattern-uri statistice, fără o înțelegere conceptuală
Interdisciplinari- tate	Filosofie, logică, viață cotidiană	Matematică, lingvistică, semiotică, cultură	LLM-urile sunt antrenate pe texte din multiple domenii, dar nu au conștiință interdisciplinară
Limbajul ca instrument	Cuvintele sunt unelte în activități umane	Cuvintele sunt puncte într-un spațiu semantic	Cuvintele sunt vectori într-un spațiu latent, folosite pentru generare și inferență
Variabilitatea sensului	Sensul se schimbă în funcție de jocul de limbaj	Sensul se modifică prin proximitate și context	Sensul este probabilistic și emergent din patternuri de co- apariție
Finalitate epistemică	Clarificarea confuziilor filozofice	Înțelegerea limbajului ca fenomen cultural și formal	Generarea de text coerent și relevant, fără intenție epistemică

Arhitectura Transformer și Sensul ca Utilizare

- Arhitectura **Transformer**, în special prin mecanismul de **Atenție (Attention)**, și **LLM-urile** bazate pe aceasta (precum GPT, BERT), nu sunt concepute filosofic, ci funcționează pe baze statistice:
- **Învățarea contextuală:** Transformerul procesează o secvență de cuvinte și folosește **mechanismul de atenție** pentru a calcula **cât de relevant** este fiecare alt cuvânt din secvență pentru sensul cuvântului curent.
- **Vectori de încorporare (Embeddings):** Cuvintele sunt reprezentate ca **vectori** într-un spațiu multidimensional. Poziția și distanța acestor vectori depind de **contextele** în care apar cuvintele în datele de antrenare masivă. Cuvintele folosite în contexte similare (adică cu sensuri similare/utilizări similare) vor avea vectori de încorporare apropiați.
- **Simularea utilizării:** Prin antrenarea pe trilioane de cuvinte și învățarea **modelelor statistice de utilizare** a cuvintelor în nenumărate contexte, modelele de tip Transformer demonstrează o capacitate remarcabilă de a genera și înțelege limbajul.

Language Inspired Computing

- Există o direcție de cercetare și dezvoltare care poate fi numită „Language Inspired Computing”, chiar dacă nu este încă un termen consacrat academic care reflectă:
 - Evoluția de la procesarea limbajului la modelarea prin limbaj.
 - Recunoașterea faptului că limbajul este o sursă de inspirație pentru arhitecturi cognitive și algoritmice.
 - O convergență între lingvistică, filozofie, semiotică și inteligență artificială.

	Exemplu de inspirație din limbaj	Aplicație în computing
Lingvistică computațională	Ambiguitate, polisemie, pragmatism	Modele de NLP care înțeleg contextul
Gramatici formale	Gramatici contextuale Marcus	Sisteme generative, parser-e adaptative
Semantica distribuțională	Proximitate semantică	Embedding-uri vectoriale (Word2Vec, BERT etc.)
Filosofie a limbajului	Wittgenstein: sensul este utilizare	Modele care învață din patternuri de utilizare
Poezie și metaforă	Transfer de sens, analogie	Generare creativă de text, storytelling AI

Language Inspired Computing

- Taxonomia

- Language-Inspired Computing**

- |

- |
 - └ **Architecture level**

- |
 - └ Attention, positional encoding, hierarchical

- |

- |
 - └ **Training level**

- |
 - └ Language modeling objective, corpora

- |

- |
 - └ **Application level**

- └ NLP tasks, semantic search, generation

Transformer-Definiție

Transformer este o arhitectură de rețea neuronală de tip sequence-to-sequence cuplat, caracterizată prin eliminarea recurenței și a convoluției în favoarea unui mecanism exclusiv bazat pe **Multi-Head Attention**. Nucleul său funcțional, **Scaled Dot-Product Self-Attention** permite modelului să pondereze dinamic și ne-local relevanța fiecărui element de intrare (token) față de toate celelalte elemente din secvență, realizând astfel o modelare contextuală paralelă de lungă distanță. Structura clasică utilizează o configurare de tip Encoder-Decoder, deși variantele generative dominante (precum GPT) operează exclusiv cu stive Decoder-Only cu mascare cauzală, fiind standardul de facto pentru construirea LLM-urilor și a Foundation Models.

Exemplu

- Fie propoziția: "*Vremea era ploioasă, iar **acest** lucru a întârziat zborul.*"
- Pentru a înțelege sensul lui "**acest**", un model RNN ar trebui să treacă prin toate cuvintele intermediare. Un Transformer, în schimb, stabilește o legătură **ne-locală** puternică (o pondere mare de atenție) direct între "**acest**" și "**ploioasă**" (sau "**vremea**") într-un singur pas, ignorând distanța fizică.

Toate LLM-urile moderne sunt Modele Fundamentale, dar nu toate Modelele Fundamentale sunt LLM-uri

	LLM (Large Language Model)	Foundation Model (Model Fundamental)
Scop Principal	Procesarea și generarea de Text .	Modelare generală, baza pentru Multiple Sarcini .
Date	Doar Text (monomodal).	Diverse (Text, Imagini, Video, Cod, etc.) (multimodal).
Relație	O Subcategorie specifică.	Termenul de Umbrelă ,mai larg.
Exemplu Simplu	GPT-3 (doar text).	GPT-4 sau DALL-E (text + alte modalități).

RAG (Retrieval-Augmented Generation)

- **RAG** este o tehnică într-o **arhitectură hibridă** care prin **ancorarea contextului la un** modul de **regăsire (retrieval)**, preia documente relevante dintr-un corpus extern (indexat adesea vectorial) și le inserează direct în **contextul *prompt-ului*** de la nivelul LLM-ului la momentul inferenței.
- Principalul său scop este acela de a **limita halucinațiile** și de a asigura **actualitatea factuală și trasabilitatea** rezultatului, fără a necesita reantrenarea modelului.

Principalele Funcții

- Generare embedding-uri(text)
- Search în documente(query_embedding)
- Construcția promptului(context, question)
- Generare răspuns(prompt)

CAG (Context-Augmented Generation)

- **CAG** este o **paradigmă largă** care cuprinde orice metodă de îmbunătățire a calității sau preciziei generării prin **manipularea *prompt*-ului de intrare** sau a **datelor adiționale** oferite modelului la momentul inferenței.
- RAG este cea mai populară metodă de CAG, dar CAG include și tehnici de **prompt engineering** avansate, cum ar fi **Chain-of-Thought (CoT)**, **Tree-of-Thought (ToT)**, sau furnizarea de exemple *few-shot* (metodă fundamentală de **CAG** care utilizează **contextul** din *prompt* pentru a ghida modelul).

Comparație dintre CAG și RAG

	CAG (Context-Augmented Generation)	RAG (Retrieval-Augmented Generation)
Definiție	Paradigmă largă de îmbunătățire a generării prin manipularea și îmbogățirea <i>prompt</i> -ului (memoria de lucru a modelului).	Tehnică hibridă specifică care utilizează un modul de regăsire extern pentru a <i>ancora factual</i> contextul de intrare.
Tipologie	Termenul umbrelă , care include multe strategii.	O sub-metodă proeminentă a CAG.
Sursa de Context	Internă la <i>Prompt</i> : Contextul este furnizat direct de utilizator (ex: exemple <i>few-shot</i> , instrucțiuni detaliate, pași CoT).	Externă la <i>Prompt</i> : Contextul este preluat dintr-o bază de date externă (vector index) și apoi injectat în <i>prompt</i> .
Scop Principal	Optimizarea raționamentului, ghidarea stilistică și controlul ieșirii (output).	Actualizarea cunoștințelor, mitigarea halucinațiilor și asigurarea trasabilității.
Flux de Lucru	1. Primește <i>prompt</i> -ul îmbogățit. 2. Generează.	1. Primește interogarea. 2. Regăsește fragmente. 3. Inserează fragmentele în <i>prompt</i> . 4. Generează.
Exemple	Few-Shot Learning, Chain-of-Thought (CoT), Prompt Engineering, Tree-of-Thought (ToT).	Sisteme bazate pe Vector Databases (e.g., Pinecone, Chroma) & LLM.
Model Modification	Zero . Doar contextul este ajustat.	Zero . Doar contextul este ajustat (la momentul inferenței).

Fine-Tuning: Definiție și scop

- **Fine-Tuning** este un proces de **transfer learning** post-pre-antrenare, în care ponderile unui model fundamental pre-existent sunt **actualizate** folosind un set de date mai mic, specific unei sarcini sau unui domeniu.
- Acesta se realizează prin **backpropagation** pe un set de date marcat (*labeled*). Variantele moderne eficiente (cum ar fi **LoRA**(Low-Rank Adaptation)/**PEFT**(Parameter-Efficient Fine-Tuning)) se concentrează pe actualizarea selectivă a unui număr redus de parametri pentru a reduce costul computațional și riscul de **catastrophic forgetting**(Apare atunci când un LLM este ajustat pe un nou set de date sau task și își suprascrie informațiile interne esențiale cu noile informații în procesul de fine-tuning)

LoRA vs PEFT

	PEFT (Parameter-Efficient Fine-Tuning)	LoRA (Low-Rank Adaptation)
Definiție	Strategie generală pentru fine-tuning eficient, actualizând doar un subset de parametri sau adăugând straturi ușoare.	O tehnică specifică din PEFT care folosește matrice cu rang redus pentru adaptarea greutăților.
Nivel	Concept umbrelă (include mai multe metode).	Implementare concretă.
Metode incluse	LoRA, Prefix Tuning, Adapter Layers, Prompt Tuning.	Doar LoRA.
Parametri antrenați	Depinde de metoda aleasă (foarte puțini comparativ cu full fine-tuning).	Extrem de puțini (doar matricele A și B cu rang redus).
Complexitate	Variabilă (unele metode sunt mai simple, altele mai complexe).	Relativ simplă și eficientă.
Cost computațional	Scăzut (comparativ cu fine-tuning complet).	Foarte scăzut (ideal pentru LLMs mari).
Flexibilitate	Mare – poți alege metoda potrivită pentru task.	Limitată la scenariile unde LoRA este aplicabilă.
Utilizare tipică	NLP, multimodal, diverse task-uri.	NLP, LLMs (GPT, BERT, LLaMA).

Formatul JSONL pentru fine-tuning

- Formatul **JSON Lines (JSONL)** pentru fine-tuning în **Azure OpenAI** variază în funcție de modelul pe care utilizatorul dorește să-l antreneze.
- Cele mai comune sunt formatul **Prompt-Completion** (pentru modelele mai vechi) și formatul **Chat** (pentru modelele mai noi precum gpt-35-turbo).
- Ambele formate necesită ca fișierul să fie o colecție de obiecte JSON, unde *fiecare obiect este plasat pe o linie separată*.

DEMO: Poveștile fraților Grimm

- Analiza personajelor din povești clasice.
- Întrebări despre impact emoțional și moral.

LangExtract: extragerea contextului și a metadatelor

- **LangExtract** este un framework conceput pentru **extragerea contextului și a metadatelor** din documente sau surse complexe, având un rol esențial în optimizarea fluxurilor **RAG (Retrieval-Augmented Generation)**, **CAG (Context-Augmented Generation)** și **Fine-Tuning**.
- **RAG**
 - **Rol:** Pregătește datele pentru indexare semantică.
 - Extrage **entități, relații, metadata** (autor, dată, sursă).
 - **Normalizează contextual(credibilitatea)** pentru a fi folosit în vector store.
 - **Avantaj:** Îmbunătățește relevanța documentelor recuperate.
- **CAG**
 - **Rol:** Creează un context augmentat, nu doar prin recuperare, ci prin **structurare semantică**
 - Organizează fragmentele extrase în **ontologii** sau grafuri de cunoștințe.
 - Permite modelelor să înțeleagă relații complexe între concepte.
 - **Beneficiu:** Răspunsuri mai coerente și explicabile.
- **Fine-Tuning**
 - **Rol:** Pregătește datele pentru antrenare eficientă.
 - Extrage **features semantice** și **tag-uri** din text.
 - Permite crearea de subseturi tematice pentru fine-tuning controlat.
 - **Beneficiu:** Reduce riscul de **Catastrophic Forgetting** prin selecție contextuală.

Avantaje LangExtract

- Automatizează **curățarea și structurarea datelor**.
- Permite integrarea cu **Azure Cognitive Search, Semantic Kernel**, sau baze vectoriale.
- Ideal pentru scenarii enterprise unde metadatele sunt critice (compliance, audit, legal).

LangExtract și ontologiile în NLP

- O **ontologie** în contextul Inteligenței Artificiale și al Web-ului Semantic este o **specificație formală și explicită a unei conceptualizări**.
Pe scurt, este un **cadru de cunoștințe** structurat care definește:
 - **Concepte (Clase)**: Entități dintr-un domeniu (ex: Persoană, Tratament, Simptom).
 - **Relații (Proprietăți)**: Modul în care conceptele se leagă între ele (ex: Persoană *are* Simptom, Medicament *tratează* Boală).
 - **Axiome**: Reguli logice care guvernează cunoștințele.

LangExtract și Ontologiile

Conexiunea se realizează prin utilizarea **ontologiei ca schemă de ieșire și ca bază de cunoștințe** pentru extracție:

- **Definirea Schemei de Ieșire:** Ontologia definește structura **obiectului JSON** pe care LangExtract trebuie să-l producă.
 - *Exemplu:* Dacă ontologia specifică un concept Pacient cu attributele nume, vârstă și relația areTratament, schema JSON (pe care LangExtract o folosește) va fi forțată să respecte această structură.
- **Îmbogățirea Semanticii (Ontology Enrichment):** LangExtract poate fi utilizat pentru a **extrage noi fapte și instanțe** din documente și a le **adăuga (popula)** în ontologie (sau într-un Graf de Cunoștințe bazat pe ontologie).
- **Validarea și Precizia Extracției:** Prin ancorarea extracției la o bază de cunoștințe formală, se reduce riscul de **halucinații** din partea LLM-ului, iar relevanța semantică a datelor extrase este mult mai mare.

Ontologii cu Azure OpenAI

Interoperabilitatea se realizează prin **API-ul de Chat Completions**

	Instrument Implicat	Acțiune
1. Configurare	Azure AI Foundry	Implementați un model GPT și obțineți URL-ul endpoint-ului și cheia de acces.
2. Apel	LangExtract (pe mașina locală)	Furnizați textul, schema JSON (ontologia) și specificați că doriți să folosiți provider-ul Azure OpenAI .
3. Extracție	LLM-ul găzduit în Azure (GPT)	Execută sarcina de extracție conform instrucțiunilor LangExtract.
4. Returnare	LangExtract	Primește datele JSON brute, le procesează, adaugă Source Grounding-ul și generează rezultatul structurat și vizualizarea.

Arhitectura sistemului propus

- 1. Preprocesare povești
- 2. Extragere trăsături (LangExtract)
- 3. Construire ontologie
- 4. Fine-tuning GPT-4.1
- 5. Integrare RAG/CAG